

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – SCENARIO BASED ASSESSMENT

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 2 – Scenario Based Assessment
Weightage:	20% of CIA	Total Marks:	100
CO2 Statement:	Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)		
Student Name:	P. Greeshma	Reg. No.:	192424418
Section / Batch:	2024- AI&DS	Date:	28/07/2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given scenario and identify the computational problem involved.
- Apply appropriate Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems wherever applicable.
- Clearly justify the chosen solution approach with proper reasoning and analytical interpretation.
- Demonstrate decision-making skills by comparing possible solutions and selecting the most suitable approach.
- Use diagrams, stepwise illustrations, or algorithmic representations wherever necessary.
- Maintain clarity, logical organization, and proper technical presentation throughout the answers.

Question Type	Focus Area	Questions
Scenario Based Assessment	Analyze real-world scenarios and apply Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems	Q1

SECTION A – SCENARIO BASED ASSESSMENT

A financial system uses Binary Search for sorted transaction records, but frequent updates disturb order. a) Explain Binary Search operation. b) Analyze the impact of unsorted data. c) Discuss maintaining sorted data for efficiency.

Code of Conduct

I P. Greeshma(192424418)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: P. Greeshma

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25%	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30%	Applies appropriate concepts effectively to solve complex realworld problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & DecisionMaking Skills	20%	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15%	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10%	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25%				
Application of Concepts	30%				
Analytical & Decision-Making Skills	20%				
Solution Design & Approach	15%				
Clarity, Justification & Presentation	10%				

Total Marks (100):	
---------------------------	--

Faculty In-charge	Course Coordinator	Head of Department
Name: _____	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

a) Explain Binary Search Operation

Binary Search is an efficient searching algorithm used to find an element in a **sorted array or list**. Instead of checking every element one by one, it repeatedly divides the search space into two halves until the required element is found.

Working Steps:

1. Find the middle element of the sorted array.
2. If the middle element is the target, the search is successful.
3. If the target is smaller than the middle element, continue searching in the left half.
4. If the target is greater than the middle element, continue searching in the right half.
5. Repeat the process until the element is found or no elements remain.

Example:

Sorted Transactions: [101, 205, 310, 425, 540, 615, 720]

Search for **540**:

- Middle = 425
- $540 > 425 \rightarrow$ Search right half
- Middle = 615
- $540 < 615 \rightarrow$ Search left half
- Middle = 540 $\rightarrow$ **Found**

Time Complexity:

- Best Case: **$O(1)$**
- Average Case: **$O(\log n)$**
- Worst Case: **$O(\log n)$**

b) Analyze the Impact of Unsorted Data

Binary Search works **only on sorted data**. If transaction records become unsorted because of frequent insertions, deletions, or updates, the algorithm cannot correctly determine which half of the data to search.

Effects of Unsorted Data:

- Binary Search may return incorrect results or fail to find existing records.
- The search process becomes unreliable because the ordering property is lost.
- A **Linear Search** may be required instead, which checks each record sequentially.
- Searching time increases from $O(\log n)$ to $O(n)$.
- Overall system performance decreases, especially when handling large financial databases.

Example:

Unsorted Transactions: [310, 101, 540, 205, 720, 425, 615]

Searching for **425** using Binary Search gives incorrect results because the list is not sorted.

c) Discuss Maintaining Sorted Data for Efficiency

To ensure Binary Search remains efficient, transaction records should always be kept in sorted order.

Methods to Maintain Sorted Data:

- Insert new transactions at the correct sorted position.
- Re-sort the records after multiple updates using efficient sorting algorithms such as Merge Sort or Quick Sort.
- Use balanced data structures like **Binary Search Trees (BSTs)** or **AVL Trees** that automatically maintain sorted order.
- Use indexed databases that preserve sorted records for faster searching.

Benefits of Maintaining Sorted Data:

- Enables Binary Search with $O(\log n)$ search time.
- Improves retrieval speed for financial transactions.
- Increases overall system efficiency and responsiveness.
- Ensures accurate and reliable search results.
- Supports fast access even for very large transaction databases.

Conclusion

Binary Search is a fast and efficient algorithm for searching sorted transaction records with **$O(\log n)$** time complexity. However, when frequent updates make the data unsorted, Binary Search becomes ineffective and searching may degrade to **$O(n)$** using Linear Search. Therefore, maintaining records in sorted order through proper insertion, sorting, or balanced tree structures is essential for ensuring efficient and accurate financial data management.

Program:

```
def binary_search(arr, key):
    low = 0
    high = len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
    return -1

arr = list(map(int, input("Enter sorted transaction records: ").split()))
key = int(input("Enter transaction to search: "))
result = binary_search(arr, key)
if result != -1:
    print("Transaction found at position:", result + 1)
else:
    print("Transaction not found.")
```

Python 3.11.9 (tags/v3.11.9:de54cf5, Apr 2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)] on win32 Type "help", "copyright", "credits" or "license()" for more information. >>> = RESTART: C:/Users/parva/assessment tool 2 daa co2.py Enter sorted transaction records: 101 205 310 425 540 615 720 Enter transaction to search: 540 Transaction found at position: 5 >>>	def binary_search(arr, key): low = 0 high = len(arr) - 1 while low <= high: mid = (low + high) // 2 if arr[mid] == key: return mid elif arr[mid] < key: low = mid + 1 else: high = mid - 1 return -1 arr = list(map(int, input("Enter sorted transaction records: ").split())) key = int(input("Enter transaction to search: ")) result = binary_search(arr, key) if result != -1: print("Transaction found at position:", result + 1) else: print("Transaction not found.")
--	--